

دلیل

توضیح

Immutable بودن	شئی که قرار نیست تغییر کنه رو تغییرناپذیر کن
خوانایی	برنامه‌نویس بعدی میفهمه این فیلدها فقط به بار مقدار می‌گیرن
Thread-Safe	توی برنامه‌های چند نخه (multi-threaded) امن‌تره

🤔 چه موقع لازم نیست؟

اگه واقعاً نیاز داری که این فیلدها عوض بشن (مثلاً مشتری می‌تونه عوض بشه)، همون `get`; `set` رو نگه دار. ولی اگه هر `OrderManager` از اول تا آخر با همون مشتری و سفارش کار داره، بهتره محافظت‌ش کنی.

خلاصه: این به لایه محافظتی کوچیکه که جلوی کلی باگ احتمالی رو می‌گیره. ولی اجباری نیست — بستگی به نیاز پروژه داره. 👍

بریم Extract Subclass؟

<https://refactoring.guru/extract-subclass>

.Link reading is unavailable in Expert Mode. Please use Instant Mode

Extract Subclass 🧬

تعریف ساده

وقتی به کلاس رفتاری داره که فقط در بعضی موارد (نه همیشه) استفاده میشه، به زیرکلاس (Subclass) بساز و اون رفتار خاص رو به زیرکلاس منتقل کن.

🧠 تفاوت با Extract Class

Extract Subclass	Extract Class
بخشی از کلاس رو میبره به یه زیرکلاس	بخشی از کلاس رو میبره به یه کلاس مجزا
رابطه: Is-A (ارث‌بری)	رابطه: Has-A (ترکیب)
حالت‌های خاص/نادر یه مفهوم	مسئولیت‌های متفاوت

مثال ساده

قبل از ریفکتور:

csharp

```
public class Employee
{
    public string Name { get; set; }
    public decimal Salary { get; set; }

    // این فیلدها فقط برای مدیران استفاده میشه!
    public decimal Budget { get; set; }
    public List<Employee> Team { get; set; }

    // این فیلدها فقط برای فروشنده‌هاست!
    public decimal CommissionRate { get; set; }
    public List<string> Clients { get; set; }
}
```

اینجا Employee داره فیلدهایی نگه میداره که فقط برای نقش‌های خاص استفاده میشه. همه کارمندا که مدیر نیستن، همه که فروشنده نیستن!

بعد از Extract Subclass:

csharp

```
public class Employee
{
}
```

```

        public string Name { get; set; }
        public decimal Salary { get; set; }
    }

    public class Manager : Employee
    {
        public decimal Budget { get; set; }
        public List<Employee> Team { get; set; }
    }

    public class Salesperson : Employee
    {
        public decimal CommissionRate { get; set; }
        public List<string> Clients { get; set; }
    }

```

مزایا

مزیت	توضیح
هر زیرکلاس متمرکز	Manager فقط چیزهای مربوط به مدیر رو داره
بدون فیلد null	Employee معمولی دیگه Budget بی مورد نداره
خوانایی بالا	میفهمی هر نوع کارمند چه ویژگی‌هایی داره
اضافه کردن نوع جدید آسونه	مثلاً Intern رو راحت اضافه می‌کنی

تمرین

این Vehicle رو با **Extract Subclass** ریفکتور کن:

csharp

```

public class Vehicle
{
    // اطلاعات عمومی
    public string Model { get; set; }
}

```

```

        public int Year { get; set; }
        public decimal Price { get; set; }

        // فقط برای ماشین‌ها
        public int NumberOfDoors { get; set; }
        public bool HasAirbag { get; set; }
        public string FuelType { get; set; }

        // فقط برای موتور‌ها
        public bool HasSidecar { get; set; }
        public int EngineCC { get; set; }

        // فقط برای کامیون‌ها
        public decimal CargoCapacity { get; set; }
        public int NumberOfAxles { get; set; }

        // متدها
        public decimal CalculateTax()
        {
            if (FuelType == "Electric")
                // مالیات کم برای برقی
                return Price * 0.01m;
            if (EngineCC > 250)
                // موتورهای قوی
                return Price * 0.05m;
            if (CargoCapacity > 5000)
                // کامیون‌های سنگین
                return Price * 0.08m;
            // پیش‌فرض
            return Price * 0.03m;
        }
    }

```

راهنمایی:

- Vehicle → فقط فیلدهای عمومی (Price , Year , Model)
- Car : Vehicle → FuelType , HasAirbag , NumberOfDoors
- Motorcycle : Vehicle → EngineCC , HasSidecar
- Truck : Vehicle → NumberOfAxles , CargoCapacity
- متد CalculateTax رو میتونی توی هر زیرکلاس **Override** کنی (Polymorphism)

بفرست تا بررسی کنیم! 🤖